

Laporan UAS Mini Project
Task/Order Tracking
Mohammad Dika Rifa Nadi
NIM : 25120500006
Kelas : Comp2

LAPORAN UAS MINI PROJECT INDIVIDU
Task/Order Tracking API — Integrasi Protokol REST, Reliability, dan Observability

Identitas	Keterangan
Nama Mahasiswa	Mohammad Dika Rifa Nadi
NIM	25120500006
Semester / Kelas	Semester 2 / Kelas Professional
Mata Kuliah	Communication Protocol
Dosen Pengampu	Haikal Shiddiq, S.Kom., M.T.
Judul Project	Task/Order Tracking API (REST) — Reliability & Observability
Link GitHub Repository	MDRNP NAP
Link Video YouTube (Unlisted)	Mohammad Dika Rifa Nadi

1. Problem Statement, Tujuan, dan Alasan Pemilihan Use Case

1.1 Problem Statement

Toko atau bisnis kecil-menengah (dicontohkan pada project ini sebagai “Toko Sinar Jaya”) umumnya masih mencatat pesanan pelanggan secara manual melalui buku catatan, chat, atau spreadsheet. Cara ini rawan menimbulkan kesalahan pencatatan status pembayaran (pending/paid), sulit dilacak riwayat perubahannya, dan tidak memiliki mekanisme perlindungan ketika terjadi lonjakan permintaan (misalnya banyak sistem yang memanggil API secara bersamaan).

Dibutuhkan sebuah layanan API yang dapat diintegrasikan oleh sistem lain (kasir, aplikasi mobile, atau workflow otomatis) untuk mencatat, membaca, dan memperbarui status pesanan secara terstruktur, disertai kontrak error yang jelas dan mekanisme reliability agar layanan tetap stabil.

1.2 Tujuan Mini Project

- Merancang dan membuktikan REST API untuk pencatatan dan pelacakan status order (Task/Order Tracking) dengan minimal 5 endpoint/aksi.
- Menguji minimal 2 skenario berhasil dan 2 skenario gagal/error pada API tersebut.
- Membuktikan penerapan pola reliability (rate limiting) melalui endpoint khusus.
- Mengumpulkan bukti observability (traffic capture Wireshark) pada level protokol HTTP/TCP.
- Menjelaskan alur teknis secara menyeluruh melalui laporan, slide, dan video demo.

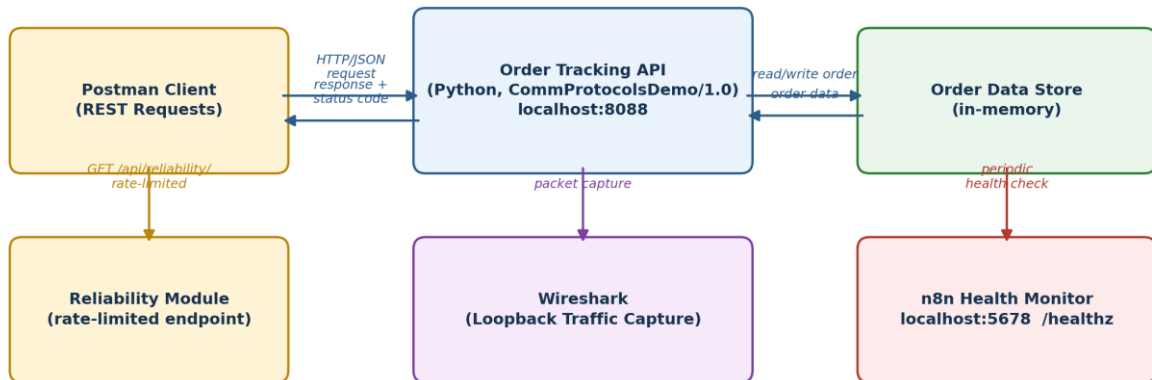
1.3 Alasan Pemilihan Use Case

Use case yang dipilih adalah No. 4 — Task/Order Tracking (REST + n8n opsional) — karena merepresentasikan kebutuhan nyata pengelolaan pesanan pelanggan mulai dari pembuatan order (status pending), pembaruan status (menjadi paid), hingga penanganan kondisi tidak normal seperti order tidak ditemukan atau permintaan berlebihan. Use case ini juga memungkinkan penerapan sekaligus tiga aspek penilaian utama mata kuliah: protocol integration (REST), reliability (rate limiting, error contract), dan observability (Wireshark, response header, status code).

2. Architecture Canvas dan Data Flow Diagram

Diagram berikut menggambarkan komponen utama project: client Postman yang mengirim request REST ke Order Tracking API (layanan Python yang berjalan di localhost:8088), API yang membaca/menulis data pada penyimpanan order, endpoint khusus reliability (rate limiting), Wireshark yang merekam traffic pada interface loopback, serta n8n yang berjalan di localhost:5678 sebagai komponen pemantauan kesehatan layanan (health check) tambahan untuk observability.

Architecture & Data Flow - Task/Order Tracking REST API



1. Client (Postman) mengirim request REST ke API (port 8088) dengan header Content-Type: application/json.
2. API memvalidasi request; jika field kurang/tidak dikenal, API membalas 400 Bad Request dengan error contract.
3. Jika valid, API membaca/menulis data pada Order Data Store, lalu membalas 200/201 dengan body JSON order.
4. Endpoint /api/reliability/rate-limited membatasi jumlah request (limit=3 / 10 detik) dan membalas 429 saat terlampaui.
5. Seluruh traffic loopback direkam Wireshark sebagai bukti observability (handshake, header, JSON payload).
6. n8n memantau endpoint /healthz secara berkala sebagai komponen observability pendukung di luar API utama.

Gambar 1. Architecture canvas dan data flow Task/Order Tracking API.

Alur data singkat: (1) client mengirim request HTTP/JSON ke API; (2) API memvalidasi field wajib dan tipe field yang diperbolehkan; (3) jika valid, API memproses ke data store dan mengembalikan response sukses (200/201) beserta representasi order; (4) jika tidak valid atau resource tidak ditemukan, API mengembalikan error contract terstruktur (400/404); (5) endpoint reliability membatasi jumlah request per jendela waktu dan mengembalikan 429 saat terlampaui; (6) seluruh traffic direkam Wireshark sebagai bukti observability, sementara n8n memantau endpoint /healthz secara terpisah.

3. Protocol Selection

Protokol utama yang digunakan pada project ini adalah REST di atas HTTP/1.1. Pemilihan REST didasarkan pada beberapa pertimbangan teknis berikut:

- Pola interaksi bersifat request-response satu kali kirim-satu kali balas (bukan streaming atau dua arah terus-menerus), sehingga tidak memerlukan protokol stateful seperti WebSocket.
- Resource pesanan (order) secara alami dapat dipetakan ke path REST: koleksi (/api/orders) dan item (/api/orders/:id), sesuai konvensi CRUD (Create, Read, Update).
- HTTP status code dapat dimanfaatkan sebagai kontrak eksplisit antara server dan client (200 OK, 201 Created, 400 Bad Request, 404 Not Found, 429 Too Many Requests), memudahkan client menentukan penanganan error secara terprogram.
- REST mudah diuji dan didokumentasikan menggunakan Postman, serta traffic-nya (karena berjalan di atas HTTP biasa pada localhost) dapat diobservasi secara transparan melalui Wireshark.
- Dibandingkan MQTT atau WebSocket, REST lebih sesuai karena tidak ada kebutuhan notifikasi real-time dua arah maupun pola publish/subscribe pada use case pencatatan order ini.

Sebagai catatan, komponen n8n (berjalan pada port 5678) digunakan secara opsional hanya untuk memantau endpoint /healthz secara berkala, bukan sebagai bagian dari alur utama pemrosesan order.

4. Endpoint / Action Design

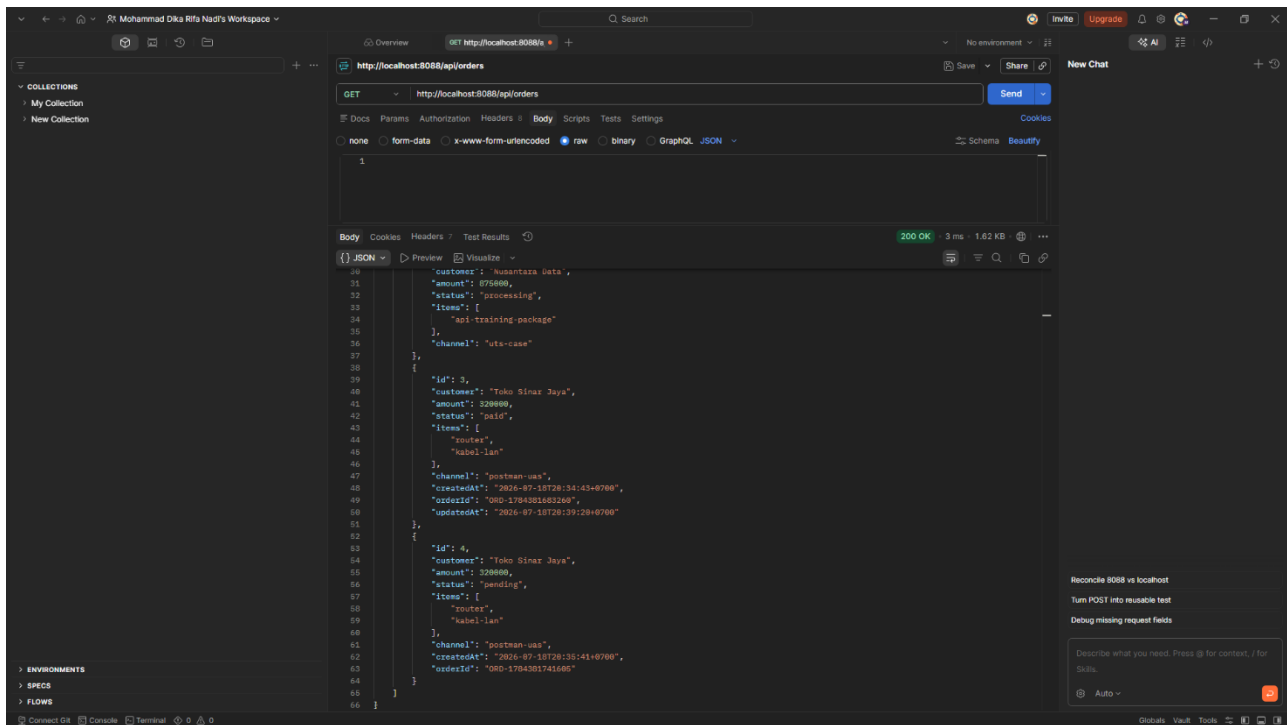
Berikut rancangan lengkap 5 endpoint yang diimplementasikan dan diuji pada project ini, mencakup method, path, header, request body, response body (ringkas), dan status code yang diharapkan.

No	Method & Path	Deskripsi	Request Body (ringkas)	Response Body (ringkas)	Status
1	GET /api/orders	Menampilkan seluruh daftar order.	–	Array objek order (id, customer, amount, status, items, channel, dsb.)	200
2	GET /api/orders/:id	Menampilkan detail satu order berdasarkan id.	–	Objek order lengkap, atau objek error jika id tidak ditemukan.	200 / 404
3	POST /api/orders	Membuat order baru.	{customer, amount, status, items[], channel}	Objek order baru berisi id, orderId, createdAt; atau error contract jika field kurang/asing.	201 / 400
4	PATCH /api/orders/:id	Memperbarui status order (mis. pending → paid).	{status}	Objek order dengan status & updatedAt terbaru.	200
5	GET /api/reliability/rate-limited	Endpoint uji reliability (rate limiting).	–	{message, remaining, limit, windowSeconds} atau {error, retryAfterSeconds, teachingPoint} saat limit terlampaui.	200 / 429

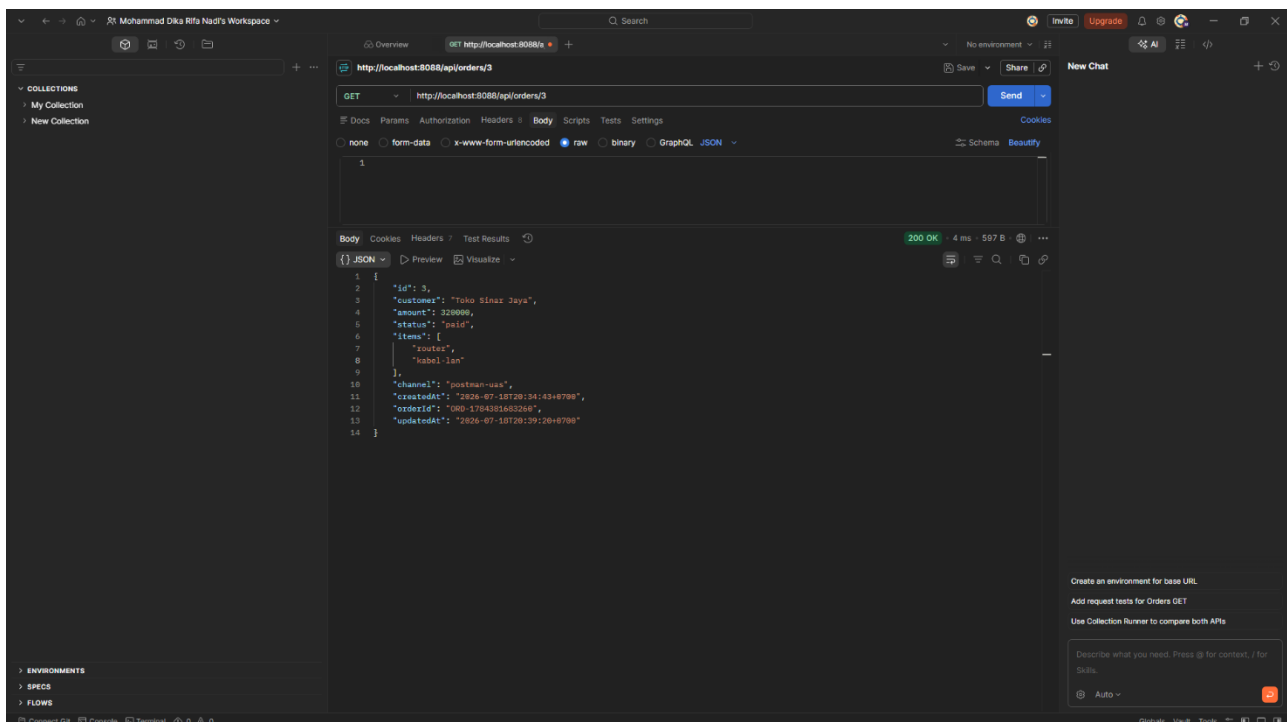
Header standar yang digunakan pada seluruh request: Content-Type: application/json. Header standar yang dikembalikan server pada seluruh response: Content-Type: application/json; charset=utf-8, Access-Control-Allow-Origin: *, Access-Control-Allow-Methods: GET, POST, PUT, PATCH, DELETE, OPTIONS, serta header Server: CommProtocolsDemo/1.0 Python/3.14.3 yang mengidentifikasi implementasi layanan.

5. Evidence Success Scenario

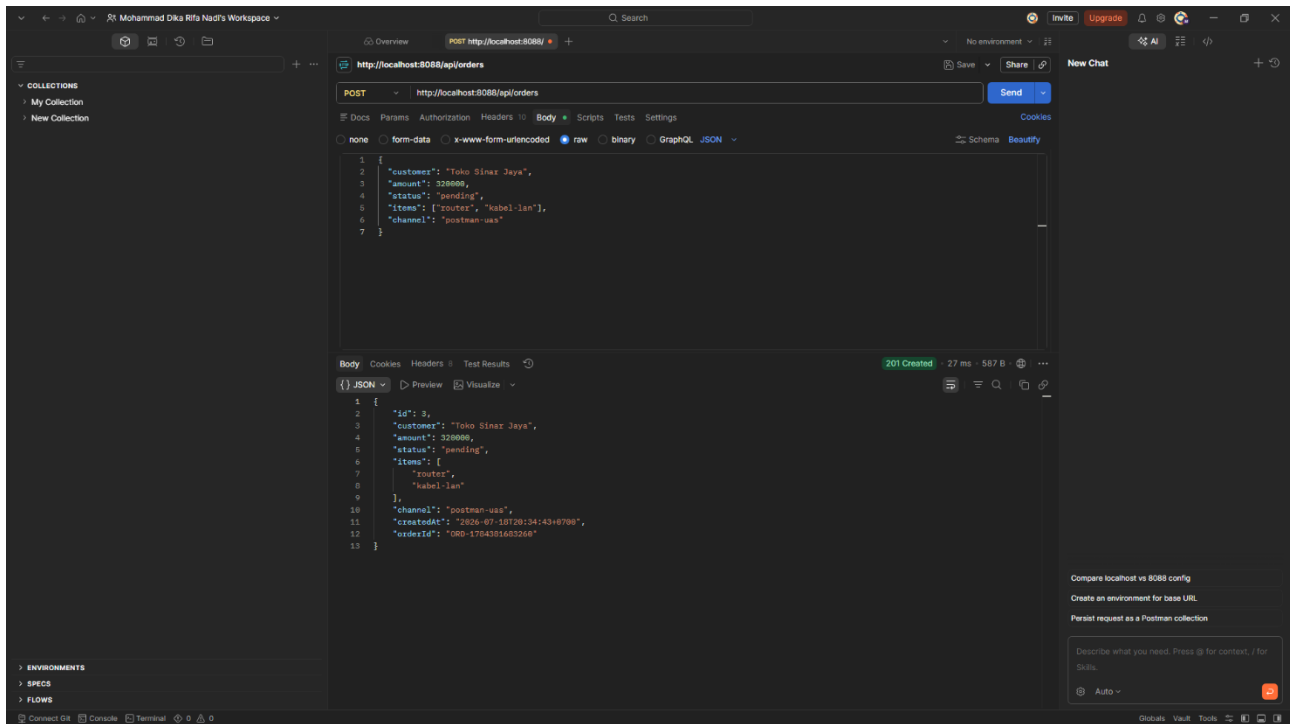
Berikut bukti pengujian skenario berhasil pada kelima endpoint, diambil dari aplikasi Postman.



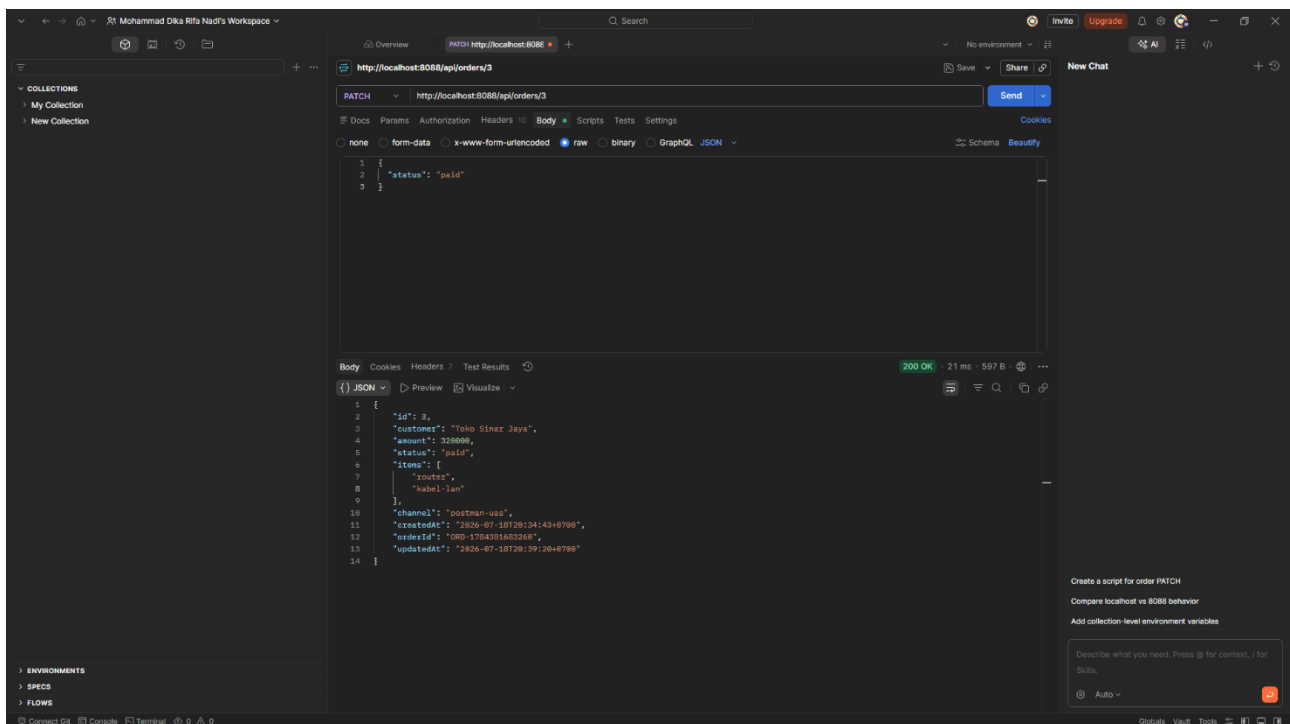
Success 1 — GET /api/orders → 200 OK.



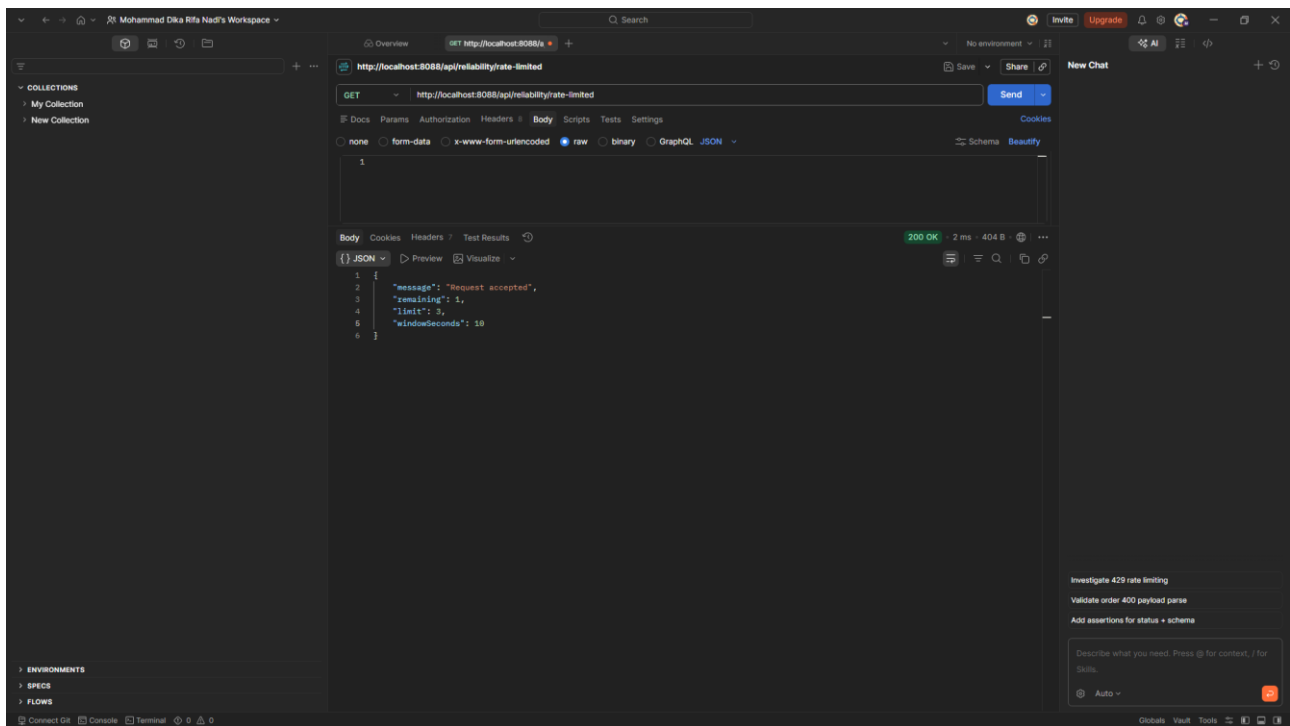
Success 2 — GET /api/orders/3 → 200 OK.



Success 3 — `POST /api/orders` → 201 Created.



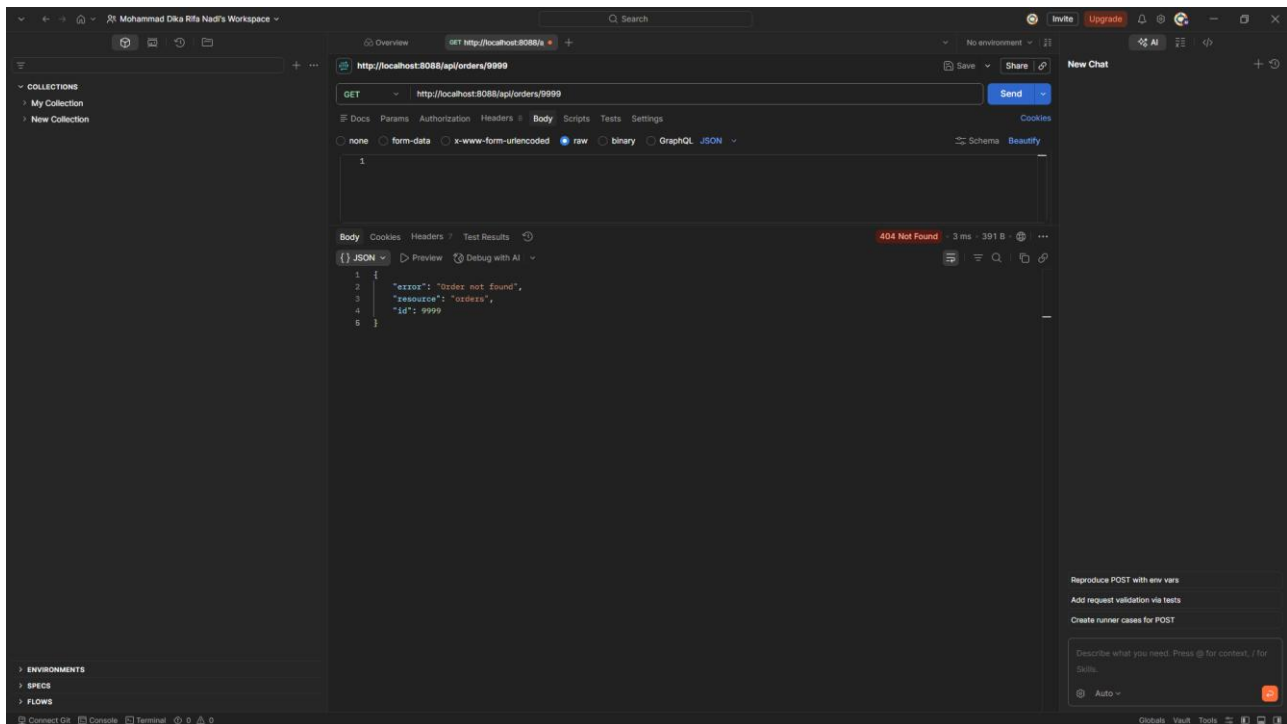
Success 4 — `PATCH /api/orders/3` → 200 OK.



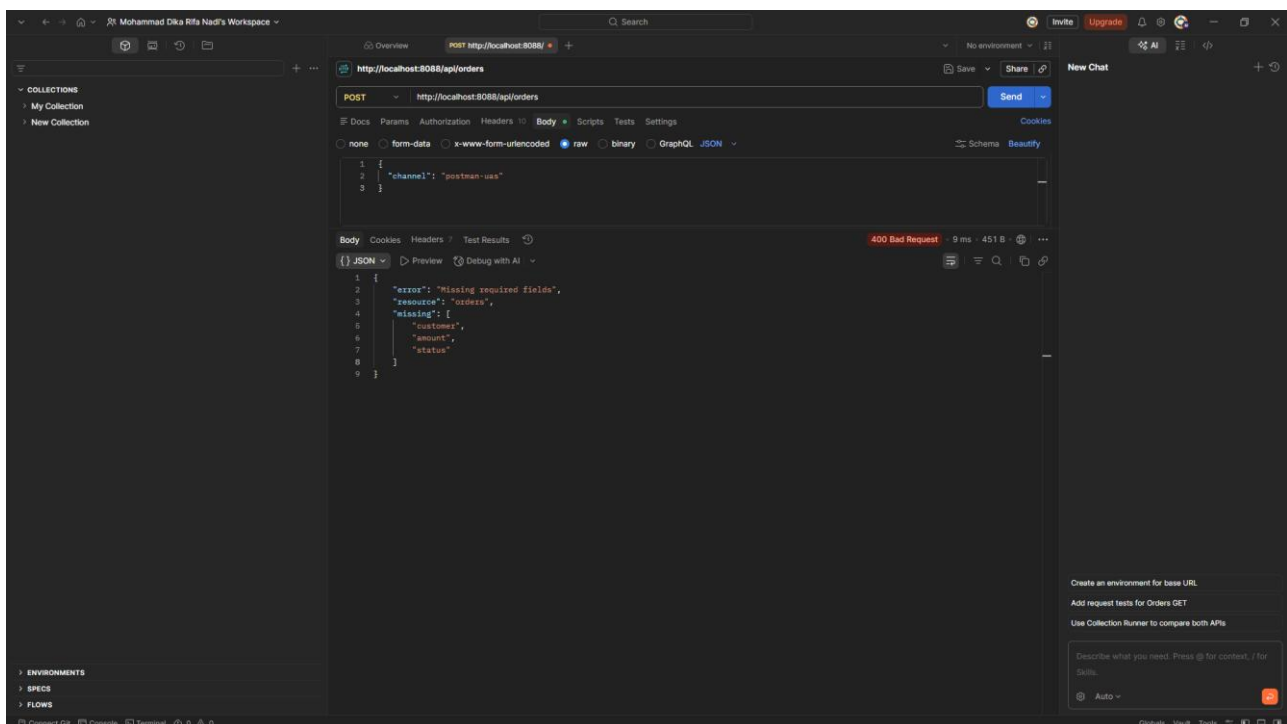
Success 5 — `GET /api/reliability/rate-limited` → `200 OK` .

6. Evidence Failure / Error Scenario

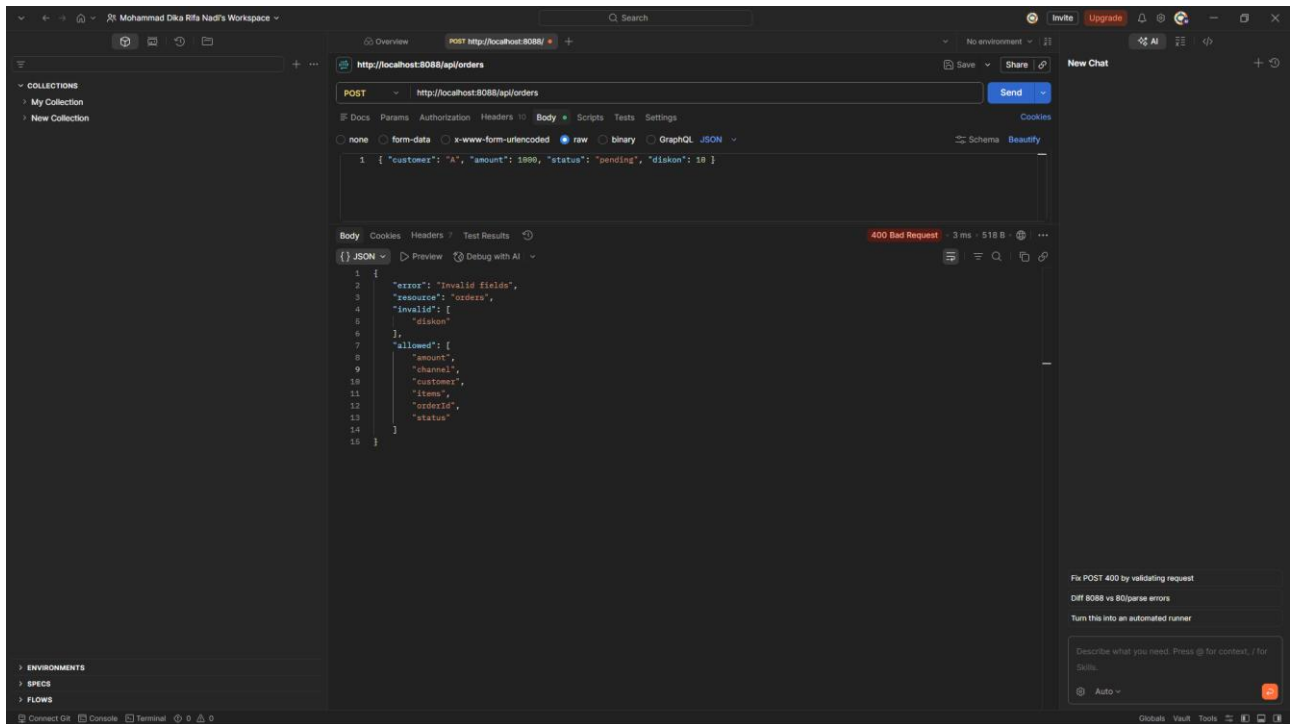
Berikut bukti pengujian skenario gagal/error yang menunjukkan bagaimana API menangani input tidak valid, resource tidak ditemukan, dan pelanggaran rate limit.



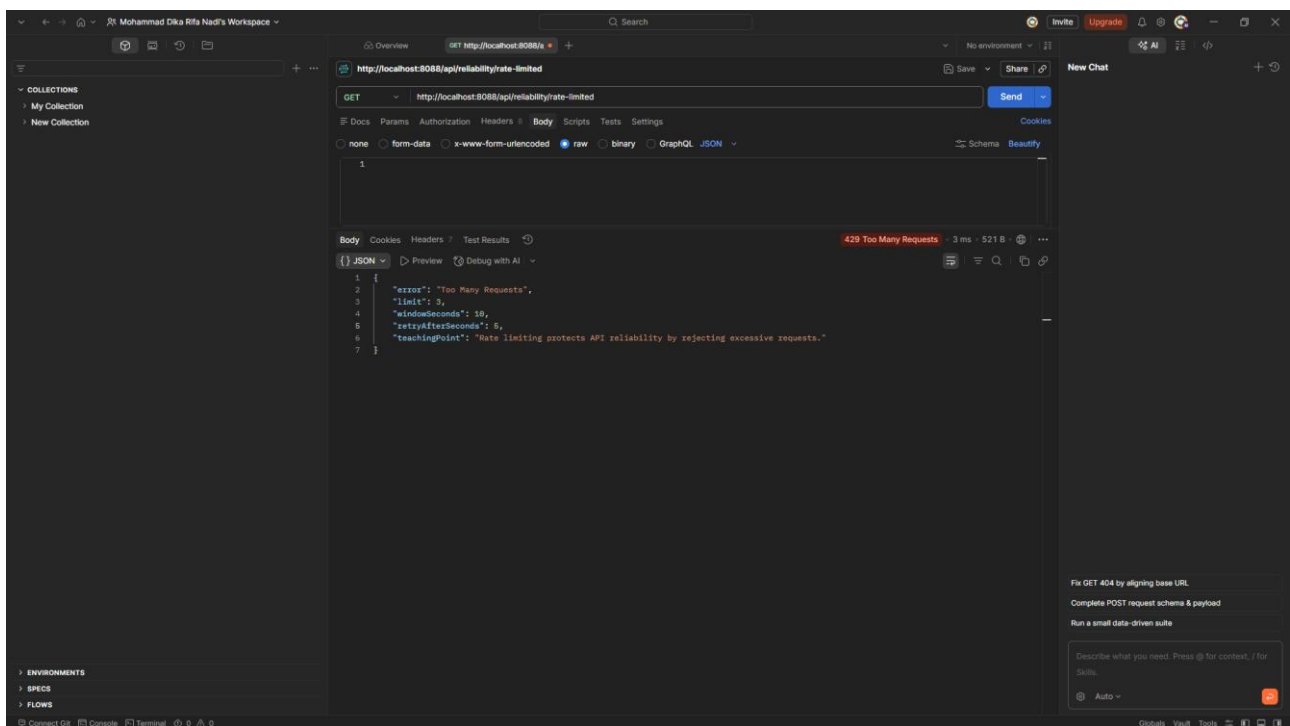
Failure 1 — GET /api/orders/9999 → 404 Not Found.



Failure 2 — POST /api/orders (field wajib tidak diisi) → 400 Bad Request “Missing required fields”.



Failure 3 — POST /api/orders (mengirim field using “diskon”) → 400 Bad Request “Invalid fields” beserta daftar allowed fields.



Failure 4 — GET /api/reliability/rate-limited (melebihi limit 3 request/10 detik) → 429 Too Many Requests, disertai retryAfterSeconds.

7. Observability dan Traffic Evidence (Wireshark)

Traffic direkam menggunakan Wireshark pada interface “Adapter for loopback traffic capture” karena seluruh komunikasi terjadi secara lokal (127.0.0.1 / ::1). Karena API berjalan di atas HTTP biasa tanpa TLS pada localhost (bukan HTTPS), payload JSON, header, maupun status code dapat terbaca sepenuhnya oleh Wireshark — hal ini dimanfaatkan sebagai bukti observability pada level protokol, sekaligus diakui sebagai keterbatasan (tidak mencerminkan trafik production yang terenkripsi TLS).

The top screenshot shows Wireshark capturing traffic on the 'Adapter for loopback traffic capture' interface. The packet list displays various TCP and HTTP traffic. The packet details pane shows an HTTP 200 OK response with a JSON body: `{\"member\": \"ok\"}`.

The bottom screenshot shows another set of captured traffic. The packet list displays various TCP and HTTP traffic. The packet details pane shows an HTTP 200 OK response with a JSON body: `{\"member\": \"ok\"}`.

Capturing from Adapter for loopback traffic capture

File Edit View Go Capture Analyze Statistics Telephony Wireless Tools Help

Capturing from Adapter for loopback traffic capture

File Edit View Go Capture Analysis Telephony Wireless Tools Help

Apply a display filter... <Ctrl>F

No.	Time	Source	Destination	Protocol	Length	Info
1498	146.107323800	127.0.0.1	127.0.0.2	DNS	65	Standard query request 0d95f1 A Cll.anydesk.com
1499	146.107552300	127.0.0.1	127.0.0.2	DNS	65	Standard query response 0d9051 A Cll.anydesk.com
1500	146.107683600	127.0.0.1	127.0.0.2	DNS	65	Standard query response 0bf8fe AAAA Cll.anydesk.com
1501	146.114603300	127.0.0.1	127.0.0.2	DNS	135	Standard query response 0b597e No such name AAAA Cll.anydesk.com SOA Ivan.n.s.cloudflare.com
1502	146.113173900	127.0.0.1	127.0.0.1	DNS	135	Standard query response 0bf8fe No such name AAAA Cll.anydesk.com SOA Ivan.n.s.cloudflare.com
1503	146.113219300	127.0.0.2	127.0.0.1	DNS	135	Standard query response 0bd853 No such name A Cll.anydesk.com SOA Ivan.n.s.cloudflare.com
1504	146.113229700	127.0.0.1	127.0.0.2	TCP	118	Destination unreachable (Port unreachable)
1505	146.115564800	127.0.0.1	127.0.0.2	DNS	135	Standard query response 0bcd7c No such name A Cll.anydesk.com SOA Ivan.n.s.cloudflare.com
1506	146.107323800	127.0.0.1	127.0.0.2	TCP	118	Destination unreachable (Port unreachable)
1507	156.769643400	:::1	:::1	TCP	90	57920 → 5078 [RST, ACK] Seq=71 Ack=1 Len=0
1508	156.786118900	:::1	:::1	TCP	64	5678 → 57920 [ACK] Seq=7 Ack=0 Win=1 Len=0
1509	156.786563900	:::1	:::1	TCP	76	54156 → 5078 [WIN] Seq=0 Win=5555 Len=0 MSS=6576 SACK_PERM
1510	156.786705600	:::1	:::1	TCP	76	5678 → 54156 [SYN, ACK] Seq=0 Ack=5555 Len=0 MSS=6576 SACK_PERM
1511	156.786708100	:::1	:::1	TCP	64	54156 → 5678 [ACK] Seq=1 Ack=1 Win=5280 Len=0
1512	156.786708100	:::1	:::1	TCP	2616	CS Localhost/Health
1513	156.789153700	:::1	:::1	TCP	64	5678 → 54156 [ACK] Seq=1 Ack=2555 Win=2976 Len=0
1514	156.792801800	:::1	:::1	MHTTP/2.	290	HTTP/1.1 200 OK [30N, application/json]
1515	156.792801800	:::1	:::1	TCP	64	54156 → 5678 [ACK] Seq=2555 Ack=2555 Win=5280 Len=0
1516	156.794181800	:::1	:::1	TCP	64	5678 → 54156 [FIN, ACK] Seq=227 Ack=2555 Win=2976 Len=0
1517	156.794181800	:::1	:::1	TCP	64	54156 → 5678 [ACK] Seq=2555 Ack=228 Win=5280 Len=0
1518	156.797318700	127.0.0.1	127.0.0.2	88	Standard query request 0cd73b AAAA localhost.anydesk.com api-seed.packetsd.io	
1519	156.797424800	127.0.0.1	127.0.0.2	DNS	88	Standard query response 0a5856 A localhost.anydesk.com api-seed.packetsd.io
1520	156.797495900	127.0.0.1	127.0.0.2	DNS	88	Standard query request 0cd73b AAAA localhost.anydesk.com api-seed.packetsd.io
1521	156.797495900	127.0.0.1	127.0.0.2	DNS	218	Standard query response 0bd72c AAAA localhost.anydesk.com api-seed.packetsd.io
1522	156.797495900	127.0.0.1	127.0.0.2	DNS	286	Standard query response 0bd72c AAAA localhost.anydesk.com api-seed.packetsd.io
1523	156.797495900	127.0.0.1	127.0.0.2	DNS	286	Standard query response 0bd72c AAAA localhost.anydesk.com api-seed.packetsd.io
1524	156.797495900	127.0.0.1	127.0.0.2	TCP	314	Destination unreachable (Port unreachable)
1525	162.893258900	:::1	:::1	TCP	64	54156 → 5678 [ACK] Seq=2555 Ack=228 Win=5280 Len=0
1526	162.893258900	:::1	:::1	TCP	64	5678 → 54156 [ACK] Seq=228 Ack=2555 Win=2976 Len=0
1527	163.845703400	127.0.0.1	127.0.0.2	DNS	71	Standard query response 0dc696 No such name txt.gettopstatus.com

Frame 1512: Captured, 2616 bytes on wire (20944 bits), 2616 bytes captured (20944 bits) on interface \Device\NPF{...}

Multi/Localhost

Internet Protocol Version 6, Src Iid: ::1, Dst Iid: ::1

Transmission Control Protocol, Src Port: 54156, Dst Port: 5678, Seq #: 5678, Len: 2554

Hypertext Transfer Protocol

Request Method: GET

Request URI: /health

Request Version: HTTP/1.1

Host: localhost:5678/r/n

Connection keep-alive/r/n

sec-ch-ua-platform: "Windows"/r/n

User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/142.0.0 Safari/537.36/r/n

Accept: */*/r/n

sec-ch-ua-mobile: ?/r/n

Accept-Encoding: gzip, deflate, br, zstd/r/n

Accept-Language: en-US,en;q=0.6,id-ID;q=0.6,sd;q=0.6,zh-CN;q=0.6

Cookie: __cf_bm=...Google Cloud...js=142...No Brand...js=...

[Response in frame 1513]

[[{"request": "http://localhost:5678/health"}]]

0000 87 55 1a 20 3a 2f 68 65 61 6c 7a 68 7a 2a 2d 84 54 get /health info

0001 56 21 25 21 00 00 48 6f 73 7a 3a 20 6c 6f 65

Selain payload, bukti observability lain yang terlihat pada capture meliputi: proses TCP three-way handshake (SYN, SYN-ACK, ACK) sebelum request dikirim, serta proses penutupan koneksi (FIN, ACK) setelah response diterima — menunjukkan siklus hidup koneksi HTTP/1.1 secara lengkap pada setiap request.

8. Testing Result dan Reliability Analysis

8.1 Ringkasan Hasil Pengujian

No	Endpoint & Skenario	Expected	Actual	Hasil
1	GET /api/orders – list order	200 OK	200 OK, array order tampil	Sesuai
2	GET /api/orders/3 – detail order	200 OK	200 OK, data order 'Toko Sinar Jaya' tampil	Sesuai
3	POST /api/orders – buat order valid	201 Created	201 Created, id & orderId ter-generate	Sesuai
4	PATCH /api/orders/3 – update status	200 OK	200 OK, status menjadi 'paid'	Sesuai
5	GET /api/reliability/rate-limited – dalam limit	200 OK	200 OK, 'Request accepted'	Sesuai
6	GET /api/orders/9999 – id tidak ada	404 Not Found	404 Not Found, error 'Order not found'	Sesuai
7	POST /api/orders – field wajib kosong	400 Bad Request	400 Bad Request, 'Missing required fields'	Sesuai
8	POST /api/orders – field asing 'diskon'	400 Bad Request	400 Bad Request, 'Invalid fields'	Sesuai
9	GET /api/reliability/rate-limited – melebihi limit	429 Too Many Requests	429, retryAfterSeconds: 5	Sesuai

8.2 Reliability Analysis

- Rate limiting: endpoint /api/reliability/rate-limited membatasi maksimal 3 request per jendela waktu 10 detik (limit=3, windowSeconds=10). Saat terlampaui, server membalas 429 Too Many Requests disertai retryAfterSeconds=5 sebagai instruksi backoff bagi client, serta teachingPoint yang menjelaskan tujuan rate limiting yaitu melindungi reliability API dari permintaan berlebihan.
- Error contract konsisten: setiap error memiliki struktur field yang seragam, misalnya {error, resource, id} untuk 404, dan {error, resource, invalid/missing, allowed} untuk 400 — memudahkan client menangani error secara terprogram tanpa perlu mem-parsing pesan bebas.
- Response time: berdasarkan pengukuran Postman, waktu respons berkisar 2–27 ms karena seluruh pengujian dilakukan pada localhost; nilai ini tidak merepresentasikan kondisi jaringan production yang sesungguhnya.
- Graceful connection handling: pada capture Wireshark terlihat siklus TCP handshake dan FIN/ACK yang tertutup rapi pada setiap request-response, menandakan tidak ada koneksi yang menggantung (hanging connection).

9. Kesimpulan, Limitation, Improvement, dan Reflection

9.1 Kesimpulan

Mini project ini berhasil merancang, menjalankan, dan membuktikan REST API Task/Order Tracking dengan 5 endpoint utama. Pengujian mencakup 5 skenario berhasil dan 4 skenario gagal/error, ditambah bukti penerapan reliability pattern (rate limiting) dan observability melalui traffic capture Wireshark pada level protokol HTTP/TCP.

9.2 Limitation

- Seluruh pengujian dilakukan pada localhost (127.0.0.1 / ::1) sehingga tidak merepresentasikan latensi maupun kondisi jaringan production yang sesungguhnya.
- Komunikasi menggunakan HTTP biasa tanpa TLS/HTTPS, sehingga tidak dapat didemonstrasikan perilaku traffic terenkripsi; hal ini diakui secara jujur sesuai ketentuan soal.
- Komponen n8n hanya digunakan sebatas pemantauan endpoint /healthz, belum diintegrasikan lebih dalam ke alur pemrosesan order (misalnya notifikasi otomatis saat status berubah).
- Data order disimpan secara in-memory (tidak persisten), sehingga data akan hilang saat service di-restart.

9.3 Improvement

- Menambahkan database persisten (mis. PostgreSQL/SQLite) sebagai pengganti penyimpanan in-memory.
- Menambahkan autentikasi (API key/JWT) dengan penanganan token yang aman.
- Mengintegrasikan n8n lebih dalam, misalnya memicu workflow notifikasi saat status order berubah menjadi 'paid'.
- Menerapkan HTTPS/TLS serta correlation ID per request untuk observability yang lebih baik pada lingkungan production.

9.4 Reflection

Melalui project ini, penulis memahami secara langsung bagaimana merancang resource REST yang konsisten, memaknai HTTP status code sebagai kontrak komunikasi, menerapkan pola reliability sederhana (rate limiting), serta pentingnya observability melalui packet capture untuk membuktikan perilaku protokol secara nyata — bukan hanya secara teori.